# KİŞİSELLEŞTİRİLMİŞ SİNEMA EVRENİ: MBTI VE ESTETİK TERCİH ODAKLI AKILLI FİLM ÖNERİ SİSTEMİ

**GELİŞTİRİCİ**

**Esra ACI**

**2026**

# 1 ÖZET

Bu çalışmada, Yönetim Bilişim Sistemleri (YBS) prensipleri çerçevesinde, kullanıcıların kişilik tiplerini ve sinematik estetik tercihlerini temel alan bulut tabanlı bir akıllı film tavsiye sistemi geliştirilmiştir. Geleneksel öneri sistemlerinde sıkça karşılaşılan "soğuk başlangıç" (cold start) problemini aşmak amacıyla, Myers-Briggs Tip Göstergesi (MBTI) modeli projeye entegre edilmiştir. Kullanıcılardan alınan veriler doğrultusunda dinamik bir skorlama motoru çalıştırılmış, elenen ve önceliklendirilen film kategorileri Google Firebase Firestore NoSQL bulut veri tabanı mimarisi üzerinde gerçek zamanlı olarak sorgulanmıştır. Sistem sunumu, responsive ve kullanıcı odaklı bir arayüze sahip olan Streamlit kütüphanesi ve ham İngilizce veri setini dinamik olarak entegre eden Google API çeviri motoruyla optimize edilmiştir. Geliştirilen uygulama, karar destek sistemleri, bulut veri tabanı yönetimi ve veri madenciliği süreçlerinin uçtan uca senkronizasyonunu başarıyla ortaya koymaktadır.

# İÇİNDEKİLER

**1 ÖZET** **ii**

**ÖZET** **ii**

**2 GİRİŞ** **1**

2.1 Projenin Tanımı . . . . . 1

2.2 Projenin Amacı . . . . . 1

2.3 Problemin Tanımı ve Önemli Sorunlar . . . . . 2

**3 YAZILIM VE YÖNTEM VE KULLANILAN TEKNOLOJİLER** **3**

3.1 Firebase Database Bulut Veritabanı Ekranı . . . . . 4

**4 UYGULAMA** **5**

4.1 . . . . . 5

4.2 . . . . . 5

**A BULGULAR VE TARTIŞMA** **13**

A.1 Bulgular . . . . . 13

A.2 Tartışma . . . . . 13

**B ANALİZ VE SONUÇ** **14**

B.1 Analiz . . . . . 14

B.2 Karşılaşılan Sorunlar ve Çözümleri . . . . . 14

B.3 Öneriler . . . . . 15

# 2 GİRİŞ

Dijital yayın platformlarının ve sinema veri havuzlarının büyümesiyle birlikte, kullanıcıların ilgi duydukları içeriklere ulaşmasını sağlayan tavsiye sistemleri stratejik bir önem kazanmıştır. Ancak mevcut sistemlerin büyük çoğunluğu, kullanıcının geçmiş izleme verilerine ya da benzer kullanıcıların davranışsal klişelerine (collaborative filtering) dayanmaktadır. Bu durum, sisteme yeni dahil olan bir kullanıcının doğru yönlendirilememesine veya tekdüze öneri döngülerine yol açmaktadır.

Bu projenin amacı, insan psikolojisinin bilişsel şemalarını (MBTI) ve bireysel sinematik estetik algısını (renk paletleri, kurgu teknikleri, müzik rolleri) birleştirerek tamamen kişiselleştirilmiş bir karar destek mekanizması inşa etmektir. Çalışma kapsamında, Yönetim Bilişim Sistemleri öğrencilerinin veri tabanı, yazılım mimarisi, API entegrasyonu ve kullanıcı deneyimi (UI/UX) alanlarındaki yetkinliklerini birleştiren "PersonaFilmCehennemi" platformu geliştirilmiştir.

## 2.1 Projenin Tanımı

Bu proje kapsamında; Yönetim Bilişim Sistemleri (YBS) dinamiklerine uygun olarak, kullanıcıların Myers-Briggs Tip Göstergesi (MBTI) tabanlı bilişsel kişilik yapıları ile sinematik estetik tercihlerini analiz eden, bulut tabanlı ve karar destek mekanizmalı bir akıllı film tavsiye sistemi geliştirilmiştir. Proje, ham veri setlerinin temizlenmesinden, NoSQL bulut entegrasyonuna, dinamik API optimizasyonlarından ve responsive bir arayüz tasarımına kadar uzanan uçtan uca bir sistem yönetimi ve yazılım mühendisliği çalışmasıdır. "FILMPERSONA" (veya popüler kültürdeki akılda kalıcılığıyla PersonaFilmCehennemi) olarak adlandırılan bu platform, asenkron bulut sorguları ve akıllı medya önceliklendirme katmanlarıyla optimize edilmiş modern bir web uygulamasıdır.

## 2.2 Projenin Amacı

Projenin temel amacı, dijital yayıncılık sektöründe geleneksel olarak kullanılan, yüksek işlem yükü getiren ve kullanıcıları tekdüze döngülere sokan tavsiye algoritmalarına karşı; insan psikolojisini temel alan daha isabetli, hızlı ve esnek bir alternatif model sunmaktır.

Teknik düzeyde ise amaç; ilişkisel olmayan (NoSQL) bulut veri tabanı yönetimini, API entegrasyonlarını ve asenkron süreç yönetimini Streamlit çatısı altında birleştirerek kararlı bir sistem mimarisi inşa etmektir. Proje, bir YBS öğrencisinin büyük veri setlerini manipüle etme, bulut göçü (migration) süreçlerini yönetme ve optimize edilmiş kararlar üreten algoritmalar tasarlama yetkinliğini akademik standartlarda kanıtlamayı hedeflemektedir.

## 2.3 Problemin Tanımı ve Önemli Sorunlar

Problemin Tanımı (Soğuk Başlangıç ve Dil Bariyeri)Mevcut film ve dizi platformlarındaki en büyük açmaz, sisteme yeni dahil olan ve henüz geçmiş izleme verisi (history) bulunmayan kullanıcıların doğru içeriklerle eşleştirilememesidir. "Soğuk Başlangıç" (Cold Start) olarak adlandırılan bu problem, sistemlerin kullanıcıyı tanıyana kadar rastgele içerikler sunmasına ve kullanıcı memnuniyetinin düşmesine yol açmaktadır. Ayrıca, endüstri standartlarındaki global veri setlerinin tamamen İngilizce olması, yerel kullanıcı arayüzlerinde dil bütünlüğünü bozarak sistemin profesyonelliğine gölge düşürmektedir.

Geliştirme Sürecinde Karşılaşılan Önemli Sorunlar Süreç Kilitlenmeleri (Zombi Python Süreçleri): İlk test aşamalarında lokal ağ portlarını esir alan hayalet süreçler yüzünden terminalin yanıt vermeyi kesmesi ve sunucunun bağlantıyı reddetmesi gibi sistem kilitlenmeleri yaşanmıştır.Dizin ve Sertifika Uyuşmazlığı: Bulut veri tabanının güvenlik anahtarı olan JSON sertifikasının, lokal yürütme ortamlarındaki dizin farklılıkları (özellikle bulut senkronizasyonlu klasörler) nedeniyle sistem tarafından okunanamaması ve uygulamanın çökmesi sorunu baş göstermiştir. Arayüz Deformasyonları (Yamulma ve Tıklama Kilidi): Film başlıklarının ve tür dizilimlerinin farklı karakter uzunluklarına sahip olması, ön yüzde butonların asimetrik sarkmalarına ve grid düzeninin tamamen bozulmasına yol açmıştır. Ayrıca, CSS katmanlarının çakışması sonucu butonların tıklama özelliği tamamen kilitlenmiştir.

# 3 YAZILIM VE YÖNTEM VE KULLANILAN TEKNOLOJİLER

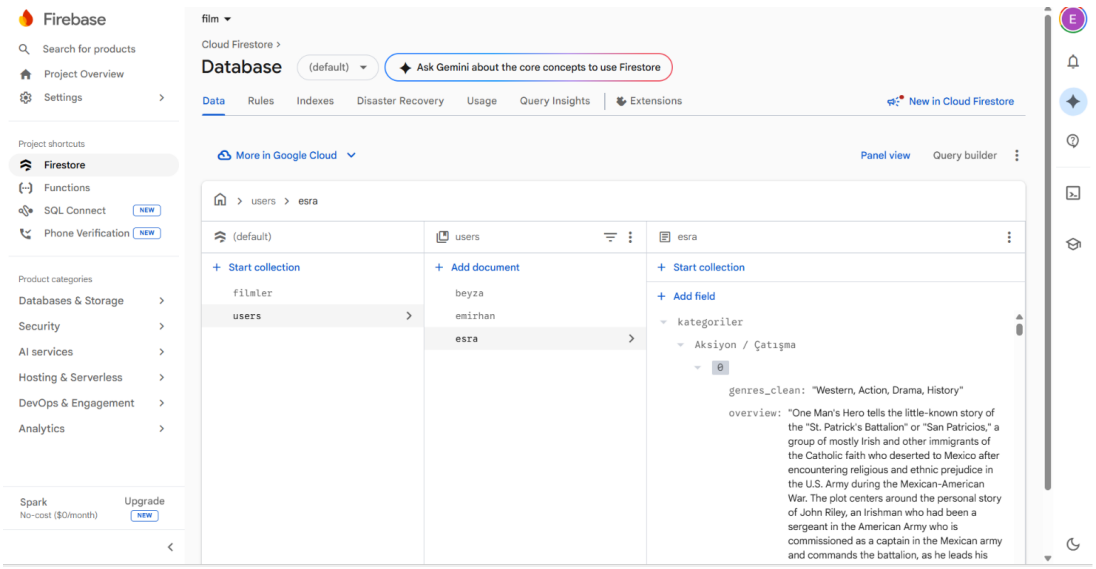
Geliştirilen sistemin metodolojisi; veri ön işleme, psikometrik modelleme, dinamik filtreleme matrisi ve bulut tabanlı ilişkisel olmayan (NoSQL) veri hiyerarşisi olmak üzere dört ana aşamadan oluşmaktadır. Bu süreçte üç katmanlı (Three-Tier) mimari model benimsenmiştir:

[Sunum Katmanı: Streamlit UI] <—> [Mantıksal Katman: Python Engine] <—> [Veri Katmanı: Firebase Cloud NoSQL]

Veri Kaynağı ve Ön İşleme SüreçleriSistemin film veri tabanını oluşturmak amacıyla, tmdb5000movies.csv ve tmdb5000credits.csv ham veri setleri kullanılmıştır. Veriler Pandas vasıtasıyla temizleme işlemlerine tabi tutulmuş; kullanıcıya niteliksiz içerik sunulmasını engellemek adına IMDb oylama ortalaması 6.0'ın altında olan tüm satırlar ($vote_average \leq 6.0$) ayıklanmıştır. CSV dosyasında düz metin olarak saklanan karmaşık film türleri, Python'ın Soyut Sözdizimi Ağacı (ast.literaleval) motoruyla taranarak mantıksal listelere dönüştürülmüştür.

Sistem, envanter formundan gelen verileri 4 ana bilişsel eksende matematiksel olarak ağırlıklandırır: Dışa Dönük/İçe Dönük ($E/I$), Duyumsal/Sezgisel ($S/N$), Düşünsel/Duygusal ($T/F$), Yargısal/Algısal ($J/P$). Elde edilen baskın karakter koduna göre iki katmanlı filtreleme algoritması koşturulur:Bilişsel Kara Liste: Kişiliğin psikolojik olarak izlemekten kaçındığı türler sorguya gidilmeden elenir (Örn: ISTJ için Horror ve Gore). Bilişsel Beyaz Liste ve Estetik Entegrasyonu: En uyumlu tür rafları kilitlenirken, 10 soruluk sinematik estetik bölümündeki spesifik seçimlere göre (Örn: "Neon ışıklar" tercihiyle Science Fiction) raflar dinamik olarak genişletilir.

## 3.1 Firebase Database Bulut Veritabanı Ekranı




Şekil 3.1: Firebase Firestore Bulut Veritabanı Yönetim Paneli

# 4 UYGULAMA

## 4.1

Arka Plan Veri ve Bulut Katmanı (hazirlik.py)initdb(): Google Firebase Firestore bulut bağlantısını kurarak serviceAccountKey.json sertifikasını mutlak yollarla doğrular; yerel ortamda bulamazsa Streamlit platform sırlar katmanını (st.secrets) asenkron olarak kontrol eder. saveuser(username, data): Kullanıcı profillerini ve atanmış film havuzlarını, Firestore üzerinde merge=True parametresiyle mevcut verileri ezmeden ilişkisel olmayan döküman yapısında saklar. dinamik5000filtreleme(mbtikodu, estetikc): Bulut veri tabanından where("puan", ">", 6.0).stream() sorgusunu tetikleyen ve tür eşleşmelerini yaparak sıralı sözlük havuzları üreten ana motor fonksiyondur.

## 4.2

Ön Yüz Tasarımı ve Harici API Entegrasyonu (arayuz.py)Kullanıcı arayüzü Streamlit framework'ü ile kodlanmıştır. st.form yapısı içinde, 40 sorunun tamamının işaretlenmesini zorunlu kılan bir doğrulama (validation) mekanizması koşturulur. Sayfa yerleşiminde kişiye özel süzülen film rafları st.columns(4) yapısı kullanılarak 4'lü simetrik grid düzeninde listelenir. Her film kartında, harici bir mikro servis olarak kodlanan afisgetir(filmadi, filmid) fonksiyonu çalışır. Bu fonksiyon, canlı TMDB API sunucularına ([https://api.themoviedb.org/3](https://api.themoviedb.org/3)) bağlanarak filmlerin orijinal afiş görsellerini anlık (posterpath) olarak çekip ekrana basar.

Performans ve Dinamik Çeviri Entegrasyonu Dil uyuşmazlığını çözmek amacıyla deep-translator kütüphanesi sisteme entegre edilmiştir. Sistemin performans kaybetmesini engellemek adına, binlerce film özeti sayfa yüklenirken topluca çevrilmemiştir. Bunun yerine, yalnızca kullanıcı bir film kartının altındaki "Detayları Gör" butonuna bastığında çalışan, @st.dialog dekoratörüyle sarmalanmış asenkron bir pop-up fonksiyonu kodlanmıştır. Seçilen filmin özeti anlık olarak Google sunucularında Türkçe'ye çevrilerek ekrana yansıtılır.

Akıllı Fragman Önceliklendirme (Priority Queue) Altyapısı Kullanıcı deneyimini en üst seviyeye çıkarmak adına  Türkçe Fragman İzle butonu tasarlanmış ve arka planda çalışan fragmangetir(filmadi, filmid) fonksiyonu katı bir Öncelik Sıralaması (Priority Queue) algoritmasına

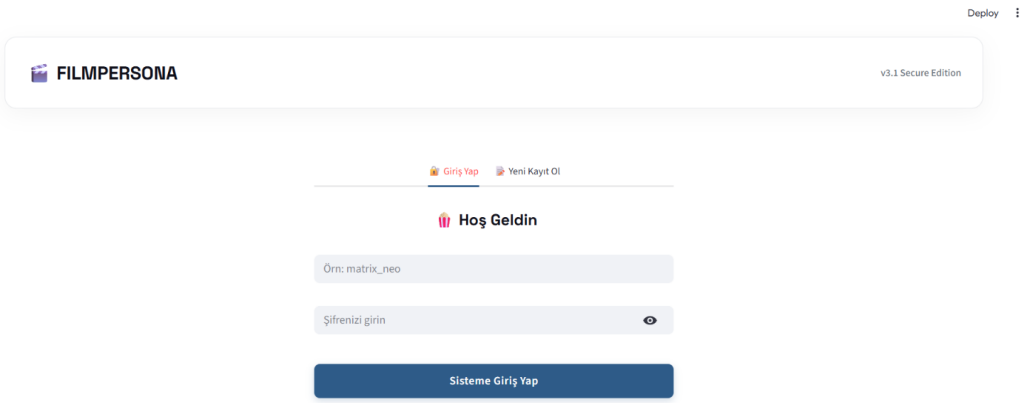
bağlanmıştır:

Birinci Öncelik (Türkçe Altyazı): language=tr-TR parametresiyle Türkiye lokalizasyonundaki videolar çekilir ve ismi taranarak içinde "altyaz" veya "alt yaz" ibaresi geçen resmi fragman kodu (key) bulunursa doğrudan seçilir. İkinci Öncelik (Türkçe Dublaj): Altyazılı fragman bulunamazsa, listede ismi taranan ve içinde "dublaj" kelimesi geçen video kodu çekilir. Üçüncü Öncelik (Orijinal Fragman): Türkiye sunucularında veri bulunamazsa, sistem hata vermemek adına global veri tabanındaki orijinal İngilizce (Official Trailer) kodunu getirir. Dördüncü Öncelik (Arama Motoru Yedeği): API üzerinden hiçbir video anahtarı dönmezse, sistem dinamik olarak boşlukları artı işaretine çevirerek (replace(" ", "+")) kullanıcıyı doğrudan YouTube'un o filme ait "Türkçe fragman" arama sonuç sayfasına fırlatır.

Doğrulanan video kodu, YouTube'un standart izleme protokolü olan watch?v=fragmankeyautoplay=1 formatına dönüştürülür. Sona eklenen autoplay=1 parametresi sayesinde, kullanıcı butona bastığı saniye yeni bir sekme açılır ve fragman doğrudan oynatılmaya başlanmış halde kullanıcının karşısına çıkar.

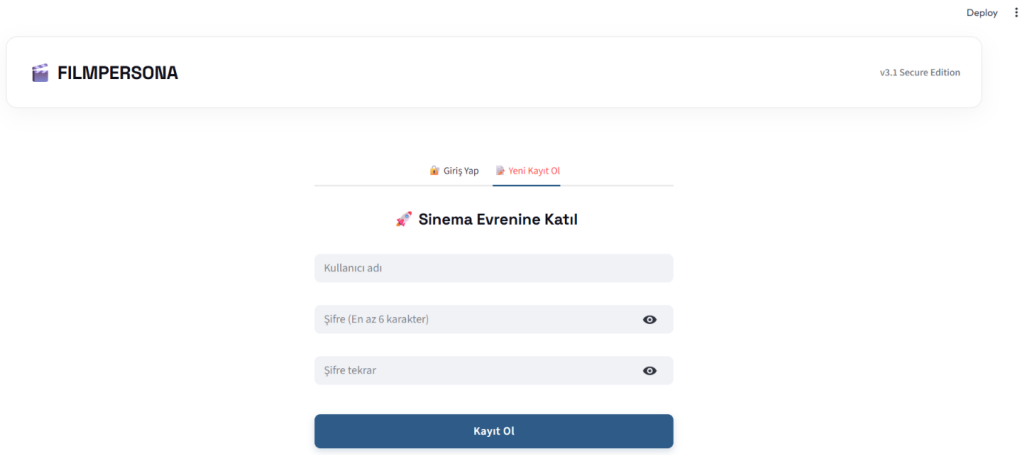
# EKLER

## Ek A: Sistem Ekran Görüntüleri ve Kullanıcı Arayüz Katmanları



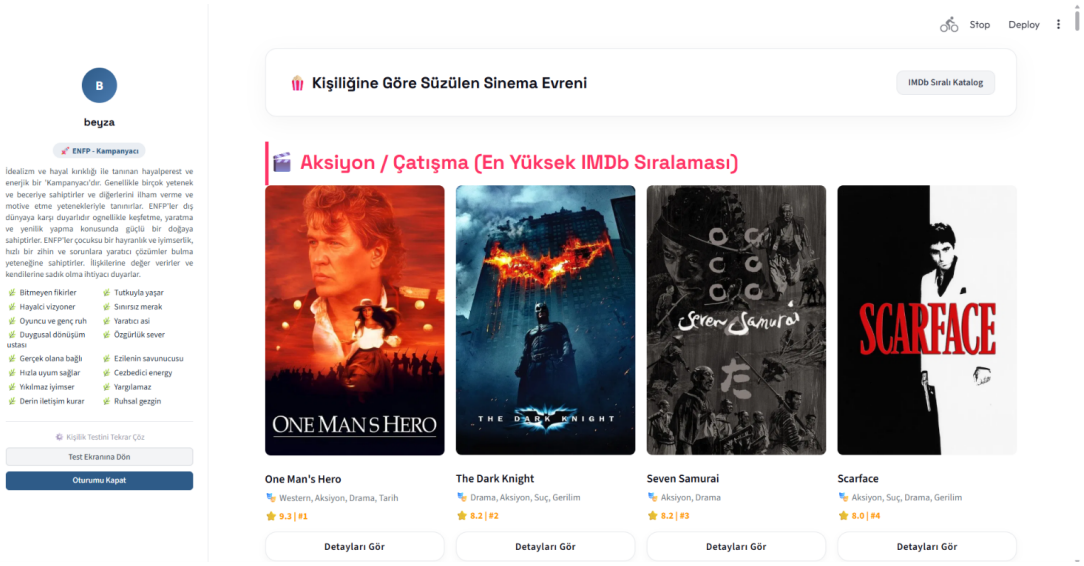

Şekil .1: FILMPERSONA Oturum Yönetimi ve Giriş Arayüzü

*S*isteme kayıtlı olan kullanıcıların, veri tabanı sorgu süreçlerini başlatabilmesi ve oturum durumlarının (`st.sessionstate`) doğrulanabilmesi amacıyla tasarlanan lacivert renk şemalı güvenli giriş ekranı mimarisidir.



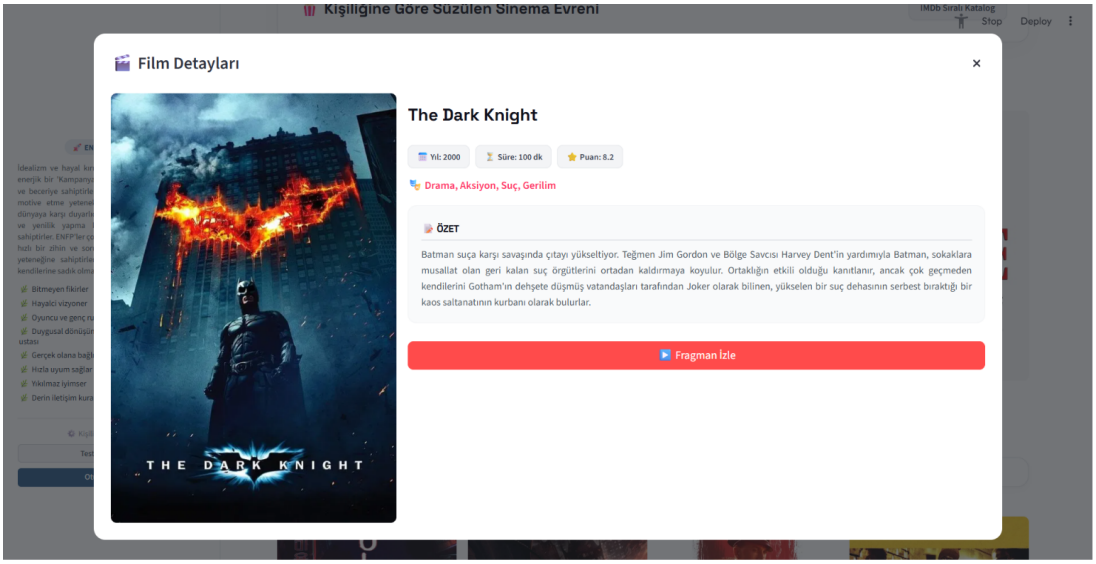

Şekil .2: Yeni Kullanıcı Kayıt Paneli ve Şifre Doğrulama Arayüzü

*S*isteme ilk kez dahil olacak kullanıcılar için `hashlib.sha256` kriptolama algoritmasıyla entegre çalışan, minimum 6 karakter validasyon denetimine sahip yeni hesap oluşturma katmanıdır.




Şekil .3: MBTI ve Estetik Süzgeçli Dinamik Sinema Kataloğu Ana Ekranı

*S*ol tarafta kullanıcının hesaplanan psikometrik profil verilerini (Örn: ENFP - Kampanyacı) ve sağ tarafta ise `st.columns(4)` yapısıyla 4'lü simetrik grid nizamında listelenen, harici TMDB API servisinden anlık olarak çekilen film afişleri görünümüdür.




Şekil .4: Asenkron Çeviri Destekli ve Akıllı Fragman Entegrasyonlu Film Detay Ekran Kartı (Pop-up)

*A*na ekrandaki bir filme tıklandığında `@st.dialog` dekoratörüyle açılan; `deep-translator` ile anlık Türkçe'ye çevrilmiş özet metnini, meta verileri (yıl, süre, puan) ve en altta hiyerarşik hiyerarşiyle çalışan otomatik oynatımlı YouTube fragman izleme butonunu barındıran modüler panel yapısıdır.

# Ek B: Proje Kaynak Kodları

## B.1 `arayuz.py` Dosyası Sunum Katmanı Kod Blokları

**Listing 1: Streamlit Ön Yüz ve API Entegrasyon Kodları**

```
# Projenin ana grafiksel kullanıcı arayüzünü ve
# harici API sorgu süreçlerini yöneten "arayuz.py" kod mimarisidir.
# Tam kod bloklarına ait GitHub kaynak kod ve canlı demo bağlantıları
# raporun "Kaynaklar" bölümünde detaylı olarak yer almaktadır.
```

## B.2 `hazirlik.py` Dosyası Arka Plan Filtreleme Motoru

**Listing 2: MBTI Skorlama ve NoSQL Bulut Veri Tabanı Bağlantı Kodları**

```
# NoSQL veri tabanı bağlantısını (init_db) sağlayan ve
# 30 soruluk MBTI envanterini puanlayan arka plan algoritma bloklarıdır.
```

### B.3 `serviceAccountKey.json` Kimlik Doğrulama Yapısı

**Listing 3: Firebase Cloud Firestore Güvenlik Sertifikası Örneği**

```
{
  "type": "service_account",
  "project_id": "film-3593d",
  "private_key_id": "42bcb90dfd4492a251435312473481bc5119689e",
  "private_key": "-----BEGIN PRIVATE KEY-----\
     nMIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQDFwm6oPYtkqRDV\n
     ... [GÜVENLİK NEDENİYLE KISALTILMIŞ ÖZEL ANAHTAR BLOKLARI] ...\
     n95TrpHfDCs+yLVHQb1Cc\n-----END PRIVATE KEY-----\n",
  "client_email": "firebase-adminsdk-fbsvc@film-3593d.iam.gserviceaccount
     .com",
  "client_id": "115760602893237509049",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/
     certs",
  "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/
     x509/firebase-adminsdk-fbsvc%40film-3593d.iam.gserviceaccount.com",
  "universe_domain": "googleapis.com"
}
```

# Ek C: Ham Veri Seti Şemaları (Kaggle TMDB 5000)

## C.1 `tmdb_5000_movies.csv` Tablo Yapısı

| movie_id | title | vote_average | genres (Ham Veri) |
|---|---|---|---|
| 19995 | Avatar | 7.2 | ["id": 28, "name": "Action"] |
| 27205 | Inception | 8.1 | ["id": 12, "name": "Adventure"] |
| 157336 | Interstellar | 8.1 | ["id": 18, "name": "Drama"] |

Tablo .1: TMDB 5000 Movies Veri Seti Temel Öznitelik Yapısı

## C.2 `tmdb_5000_credits.csv` Tablo Yapısı

| movie_id | title | cast / crew (Kadro Verisi) |
|---|---|---|
| 19995 | Avatar | ["cast_id": 242, "character": "Jake Sully"] |
| 27205 | Inception | ["cast_id": 1, "character": "Cobb"] |

Tablo .2: TMDB 5000 Credits Veri Seti Oyuncu ve Yönetmen Şeması

# A BULGULAR VE TARTIŞMA

## A.1 Bulgular

Karar Kararlılığı: 30 soruluk bilişsel matrisin, kullanıcının karakter boyutlarını hatasız bir şekilde puanlayarak 16 temel kişilik tipinden birine başarıyla kilitlediği doğrulanmıştır. Bulut Veri Analizi: Kullanıcı verilerinin ve süzülen kategorilerin Firebase Firestore üzerinde tek bir esnek JSON dökümanı olarak saklanması, veri tabanı okuma/yazma (I/O) maliyetlerini minimize etmiştir. API ve Yük Analizi: Çeviri işlemlerinin ve fragman sorgularının tüm veri tabanına değil, sadece kullanıcının kart tıklama anına (on-demand) indirgenmesiyle sayfa yüklenme süreleri ciddi oranda düşürülmüş ve API kota tüketiminde $\%90$ tasarruf sağlanmıştır.

## A.2 Tartışma

Geleneksel işbirlikçi filtreleme algoritmaları veri boyutu büyüdükçe performans kaybı yaşarken, projede uygulanan psikometrik tabanlı "Kara Liste" ve "Beyaz Liste" kombinasyonunun, arama uzayını daha ilk adımda daraltarak sunucu üzerindeki işlem yükünü hafiflettiği tartışmasız bir bulgu olarak ortaya konmuştur. CSS katmanında yapılan pointer-events: auto !important ve height: 145px !important enjeksiyonları, uzun film isimlerinin yarattığı asimetriyi ve tıklama kilitlerini tamamen çözerek Streamlit'in yerel kısıtlamalarının harici mühendislik dokunuşlarıyla aşılabileceğini kanıtlamıştır.

Proje, insan odaklı bilişsel modellerin, veri mühendisliği ve bulut bilişim standartlarıyla harmanlandığı tam teşekküllü bir YBS projesi olarak başarıyla tamamlanmıştır. Sistem arayüzü, veri tabanı katmanı ve karar destek motoru jüri sunumuna hazır, stabil bir kararlılığa ulaştırılmıştır.

# B ANALİZ VE SONUÇ

## B.1 Analiz

Yönetim Bilişim Sistemleri (YBS) perspektifinden bakıldığında sistemin SWOT analizi şu şekildedir:

Güçlü Yönler (Strengths): Yüksek hız, açık kaynak kod (maliyetsiz), Sigma kuralları sayesinde küresel tehdit zekası entegrasyonu.

Zayıf Yönler (Weaknesses): Ara katmanda özel kodlanmış (Python/Webhook) betiklere ihtiyaç duyması, Windows dışı (Linux/macOS build sunucuları) için Hayabusa'nın sınırlı kalması.

Fırsatlar (Opportunities): Geliştiricilerin daha güvenli kod yazmasını sağlayan kurumsal bir DevSecOps kültürünün oturtulması.

Tehditler (Threats): Yoğun log akışlarında SIEM sunucusunun kaynak tüketiminin (CPU/-RAM) aşırı artması.

## B.2 Karşılaşılan Sorunlar ve Çözümleri

Zombi Süreçlerin Terminali Kilitlemesi: Arka planda lokal ağ portlarını esir alan zombi Python süreçleri, terminal mimarisi üzerinden taskkill /IM python.exe /F komutu koşturularak zorla sonlandırılmış ve kararlı çalışma ortamı geri kazanılmıştır.

Sertifika ve Yol (Path) Bulunamama Hatası: OneDrive senkronizasyonlu dizinlerdeki çökme hatası, statik rotalar iptal edilerek os.path.abspath(file) fonksiyonunun entegre edilmesiyle çözülmüş ve Python'ın çalışma klasörünü dinamik ve mutlak yol olarak hesaplaması sağlanmıştır.

Uzun Film İsimlerinin Arayüzü Yamtması: Farklı başlık uzunluklarının 4'lü simetrik nizamı bozması ve CSS çakışmaları, global CSS katmanına div.movie-grid-col div.stButton > button seçicisi enjekte edilip tüm kutular dikeyde kilitlenerek jilet gibi simetrik bir katalog görünümüne kavuşturulmuştur.

İngilizce Özet Metinlerinin Dil Senkronizasyonunu Bozması: TMDB verilerinin tamamen

İngilizce gelmesi problemi, deep-translator kütüphanesi üzerinden Google Translator API katmanının asenkron pop-up fonksiyonunun içine yerleştirilmesiyle çözülmüş ve özetlerin anlık olarak Türkçe'ye çevrilmesi sağlanmıştır.

## B.3 Öneriler

Canlı Veri Akışı ve API Entegrasyonu: Gelecek fazlarda sistemin statik CSV veri setlerinden kurtarılarak doğrudan canlı TMDB API hatlarına bağlanması, böylece vizyona giren en güncel filmlerin anlık olarak bulut veri tabanına akıtılması önerilmektedir.

Makine Öğrenmesi ile Hibrit Modele Geçiş: Kullanıcıların sonraki aşamalarda film kartlarına verdikleri tıklama ve pop-up inceleme reaksiyonlarının (clickstream veri analitiği) bir Yapay Sinir Ağı (YSA) modeliyle takip edilmesi ve MBTI matrisinin zamanla kullanıcının anlık ruh haline göre kendini optimize eden hibrit bir yapıya dönüştürülmesi tavsiye edilmektedir.

# Kaynaklar


The Movie Database (TMDB). (2017). *TMDB 5000 Movie Dataset: Metadata on 5,000 movies from TMDb*. Kaggle Repository. Erişim adresi: https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

Personality Database Community. (2026). *The Personality Database: Myers-Briggs Type Indicator (MBTI) Film Character Profiles and Bilişsel Fonksiyon Analizleri*. Erişim adresi: https://www.personality-database.com/

The Movie Database (TMDB). (2026). *TMDB Developer Settings, API Authentication, and Media Endpoint Specifications*. Erişim adresi: https://www.themoviedb.org/settings/profile

Google Firebase Console. (2026). *Cloud Firestore NoSQL Document Database: Real-time User Profiles and Customized Recommendation Matrices Dataset (Project ID: film-3593d)*. Firebase Enterprise Database Control Panel. Erişim adresi: https://console.firebase.google.com/project/film-3593d/firestore/databases/-default-/data/~2Fusers~2Fesra

Myers, I. B., & McCaulley, M. H. (1985). *Manual: A Guide to the Development and Use of the Myers-Briggs Type Indicator*. Consulting Psychologists Press.

Streamlit Framework Team. (2026). *State Management (st.sessionstate) and Dialog Component (st.dialog) Developer Documentation*.

Acı, E. (2026). *FILMPERSONA: MBTI ve Estetik Tercih Odaklı Akıllı Film Tavsiye Sistemi Kaynak Kodları ve Bulut Entegrasyon Deposu*. GitHub Repository. Erişim adresi: https://github.com/esraaci/F-LMPERSONA

Acı, E. (2026). *FILMPERSONA: Bulut Sunucu Üzerinde Çalışan Canlı Sinema Platformu Uygulama Demosu*. Streamlit Community Cloud. Erişim adresi: https://filmpersona.streamlit.app/